

CC91 校园论坛系统 — 项目总结报告

项目周期: 2026 年 4 月 – 2026 年 6 月 团队规模: 11 人 代码提交: 156 commits (当前分支)
版本: v2.0 (微服务架构)

一、项目概述

CC91 是一个面向校园场景的现代化论坛系统，采用 Spring Boot 微服务架构 + React 19 前端，支持用户认证、帖子管理、评论互动、通知推送、管理后台等完整功能闭环。

经过多轮迭代，项目从单体架构演进为 **7 个微服务 + API 网关 + 服务注册中心** 的分布式系统，并通过 Docker Compose 实现一键部署。

1.1 架构演进时间线

时间	阶段	关键产出
2026-04	单体基线	Spring Boot 单体后端 + React 19 前端，用户/资料模块完成 (M1/M2)
2026-04-13	AI 协作体系建立	Lead/Developer/QA 三类智能体协议 (CLAUDE.md / developer.md / qa.md)，4 个 Skills 落地
2026-05	测试补齐	后端 JUnit 5 + MockMvc 单元/集成测试体系，前端 Vitest + RTL 组件测试
2026-05-28	CodeGraph MCP 接入	引入代码知识图谱，加速符号搜索与调用链追踪
2026-06-09 ~ 06-10	微服务拆分	单体 → Eureka + Gateway + 6 业务微服务，OpenFeign + Resilience4j 通信
2026-06-11 ~ 06-12	容器化	Docker Compose 编排 9 个容器，多阶段构建，healthcheck 启动顺序
2026-06-13 ~ 06-14	监控与压测	Prometheus + Grafana + 5 条告警规则，万级数据下 8 场景压测

二、已实现的功能

2.1 核心功能矩阵

模块	功能	状态
用户系统	注册 / 登录 / 退出 / 邮箱验证 / 密码重置	✓
	JWT 双 Token 机制 (Access + Refresh)	✓
	账户锁定 (多次失败触发, 阈值由后端配置控制)	✓
	个人资料编辑、头像上传	✓
论坛核心	版块分类浏览	✓
	发帖 (支持草稿/发布双状态)	✓
	帖子编辑、删除 (权限控制)	✓
	评论 + 多级嵌套回复	✓
	点赞 / 收藏 (toggle 机制)	✓
	全文搜索	✓
通知系统	评论/回复实时通知	✓
	通知已读/全部已读	✓
	未读数角标	✓
管理后台	用户管理 (封禁/解封)	✓
	分类管理 (增删改排序)	✓
	公告管理 (发布/置顶)	✓
	内容审核 (举报处理)	✓
安全性	bcrypt 密码哈希	✓
	XSS 双层防护 (DOMPurify + HtmlSanitizer)	✓
	RBAC 权限控制	✓
	JWT 无状态认证	✓
	文件上传多层校验	✓

2.2 页面与路由

路径	页面	认证
/	首页 (公告 + 版块 + 最新帖子 + 热门)	否
/login	登录	否
/register	注册	否
/category/:id	版块帖子列表	否
/posts/:id	帖子详情 + 评论	否
/search	搜索结果	否
/profile/:username	个人主页	否
/profile/edit	编辑资料	是
/notifications	通知列表	是
/admin/*	管理后台	管理员
/announcements/:id	公告详情	否

三、掌握的关键技术

3.1 技术栈总览

层级	技术选型	说明
后端框架	Spring Boot 3.2.12	Java 17, Maven
微服务治理	Spring Cloud 2023.0.0	Eureka + Gateway + OpenFeign
安全框架	Spring Security + JWT (jjwt 0.12.3)	无状态认证 + RBAC
数据库	MySQL 8.0.46	Flyway 9.x 版本化管理
前端框架	React 19 + Vite + TypeScript	函数组件 + Hooks
状态管理	@tanstack/react-query	服务端状态缓存
断路器	Resilience4j 2.1.0	熔断、降级、重试
监控	Spring Boot Actuator + Micrometer + Prometheus + Grafana	全链路指标采集
容器化	Docker + Docker Compose	9 个容器一键部署
测试	JUnit 5 + MockMvc + Vitest + RTL	分层测试
前端安全	DOMPurify	XSS 前端层防护
后端安全	HtmlSanitizer + BCrypt	XSS 后端层防护 + 密码哈希

3.2 智能技术应用 (Claude Code + MCP Server + Skills)

本项目在开发过程中深度应用了 AI 辅助开发技术，形成 "Agent + MCP + Rules + Skills" 的完整协作闭环。

3.2.1 Claude Code 作为开发平台

- **角色**：Claude Code 担任项目 Lead，通过 `CLAUDE.md` 定义完整的工作协议（团队、5 阶段 workflow、审查清单、技术栈声明）
- **Subagent 体系**：
 - **Lead**（项目根 `CLAUDE.md`）：需求理解、任务拆解、代码审查、最终验收
 - **Developer Agent**（`.claude/agents/developer.md`）：全栈开发，配备 5 个专业技能
 - **QA Agent**（`.claude/agents/qa.md`）：测试驱动，强制运行测试并输出报告
- **工作流**：Phase 1 理解规划 → Phase 2 开发 → Phase 3 测试 → Phase 4 审查 → Phase 5 收尾

3.2.2 MCP Server (Model Context Protocol)

MCP 是 Anthropic 提出的开放协议，为 AI 智能体提供连接外部工具和数据源的标准接口。本项目配置的 MCP Server：

MCP Server	用途	实际应用
CodeGraph	基于 Tree-sitter 的代码知识图谱，提供亚毫秒级符号搜索	代码导航（codegraph_context 一键获取任务上下文）、调用追踪（codegraph_trace 跟踪完整调用链）、影响分析（codegraph_impact 评估变更影响范围）。项目共索引 308 个文件，4284 个符号，6998 条边，替代传统 grep/read 循环
Playwright	浏览器自动化	QA 智能体的 E2E 测试与页面视觉验证，动态决策"点击哪个按钮、检查哪个元素"
IDE (executeCode / getDiagnostics)	Jupyter Kernel 执行 + VS Code 诊断	在 Jupyter 中执行 Python 压测脚本，获取语言服务器诊断信息

选用依据：Bash 可以运行预编写脚本，但无法在运行时动态决策。CodeGraph 让 AI 像 IDE 一样理解代码结构（函数→调用者→被调用者→影响范围），Playwright 让 QA 像真人一样"看着页面操作"。文件操作、构建测试、Git 管理等日常任务 Claude Code 内置工具已完全胜任，避免过度工程化。

3.2.3 规则系统 (Rules)

规则系统约束 AI 的输出边界和行为规范：

规则文件	约束范围
CLAUDE.md (项目根目录)	Lead 智能体的工作协议：团队定义、5 阶段 workflow、审查清单、技术栈声明
.claude/agents/developer.md	Developer 智能体行为约束：技术栈、核心职责、工作原则、项目结构约定
.claude/agents/qa.md	QA 智能体行为约束：测试质量红线（禁止测试框架行为、必须测试业务分支）、测试优先级、报告格式

选用依据：将团队经验固化为可执行约束。例如 QA 规则明确禁止"测试框架行为"（如 @Valid 注解、Bean 注入），强制要求"测试业务逻辑分支"（如登录失败计数、账户锁定触发），直接提升测试有效性。

3.2.4 技能 (Skills) 体系

技能模块将领域最佳实践沉淀为可复用的知识单元。项目配置了 5 个 Skills：

Skill	来源	用途
java-springboot	社区开源	Spring Boot 最佳实践：项目结构、依赖注入、Web 层、数据层、日志、测试、安全
springboot-patterns	社区开源 (ECC)	Spring Boot 架构模式：REST API 设计、分层服务、DTO、缓存、异步、限流、错误处理、可观测性
typescript-react-reviewer	社区开源	TypeScript + React 19 代码审查：反模式检测、状态管理、React 19 Hook 陷阱、类型安全
vercel-react-best-practices	Vercel 官方	React 性能优化 60+ 条规则：异步瀑布、Bundle 优化、服务端性能、重渲染、渲染策略
web-design-guidelines	社区开源	Web 界面设计规范

选用依据：技能的可组合性使得"一个 Developer 智能体 + 多个 Skills"比"多个 Developer 智能体"更具经济性和可维护性。技能通过 `skills-lock.json` 版本化锁定，保证旧流程可回溯。

四、开发流程与团队协作

4.1 开发流程

需求分析 → 任务拆分 → Developer 实现 → QA 测试 → Lead 审查 → Git 提交

更详细的 5 阶段工作流：

阶段	Lead	Developer	QA
Phase 1 理解规划	与用户对话、TaskCreate 编排依赖	—	—
Phase 2 开发	SendMessage 分配任务 (含验收标准)	阅读现有代码、参考 Skills、实现代码、自测	—
Phase 3 测试	SendMessage 分配测试任务	—	运行 <code>mvn test</code> / <code>npm test</code> 、按 qa.md 报告格式输出
Phase 4 审查	按审查清单检查 (安全/API/错误/质量)	修复阻断问题	补充回归
Phase 5 收尾	确认任务完成、提交变更	—	—

4.2 团队角色分工 (11 人)

成员	软件工程角色	主要职责	负责维护的智能体配置
徐哲楷	项目负责人 / 产品统筹	控制范围、确认里程碑、统一验收口径	Lead 智能体 (CLAUDE.md 协议、审查清单)
蔡致	前端工程师	页面结构、路由、表单、交互实现	Developer + typescript-react-reviewer
刘一鸣	前端工程师	状态管理 (React Query)、接口联调、错误处理	Developer + vercel-react-best-practices
项方灿	前端工程师	样式系统、响应式适配、可用性优化	Developer + vercel-react-best-practices
王建皓	后端工程师	登录、注册、鉴权、会话管理 (Spring Security + JWT)	Developer + springboot-patterns
李明睿	后端工程师	帖子、评论、内容治理接口	Developer + springboot-patterns
罗新鹏	后端工程师	数据模型 (JPA Entity + Flyway 迁移)、校验与异常处理	Developer + java-springboot
朱城弘	测试工程师	测试用例设计、后端接口测试 (JUnit 5 + MockMvc)	QA + qa.md 规则维护
陈瑜凡	测试工程师	前端组件测试 (Vitest + RTL)、回归验证	QA + qa.md 规则维护
陈元煦	DevOps 工程师	环境配置、构建发布、数据库运维、Docker 容器化	自动化脚本 (Maven/npm/Docker)
王帆	安全与审查工程师	安全基线审查、代码审查、知识沉淀	CLAUDE.md 审查清单 + 安全规则

4.3 协作机制

- **任务驱动**：每个任务包含唯一 ID、背景目标、实现范围、验收标准、依赖关系
- **阻塞升级**：任务阻塞超 30 分钟自动上报，超 2 次未解决则由 Lead 调整策略
- **问题跟踪**：`docs/issues.md` 实时维护未修复 Bug，P0/P1/P2 优先级管理
- **代码审查**：每次提交前通过安全检查、API 设计、错误处理、代码质量、测试质量五大维度审查

4.4 Git 工作流

- **分支策略**：`main` → `develop` → `feat/*` / `fix/*`

- **提交规范:** Conventional Commits (`feat:` / `fix:` / `refactor:` / `test:` / `docs:` / `chore:`)
- **PR 流程:** 功能分支 → `develop` , 通过审查后合并
- **AI 产物约束:** AI 生成代码必须绑定到具体任务编号, 不允许直接合并, 必须经过人工确认

五、个人心得体会

说明: 本章为 11 位成员的心得占位结构, 具体内容待用户收集齐素材后回填。

5.1 徐哲楷 — 项目负责人 / Lead 协议维护

承担角色: 项目负责人 (统筹) + Lead 智能体协议 (CLAUDE.md) 维护

主要挑战:

待补充

解决方法:

待补充

收获与反思:

待补充

5.2 蔡致 — 前端工程师 (页面结构、路由、表单)

承担角色: 前端工程师, 负责页面结构、路由设计、表单交互; 维护 Developer 智能体 + typescript-react-reviewer 技能

主要挑战:

待补充

解决方法:

待补充

收获与反思:

待补充

5.3 刘一鸣 — 前端工程师（React Query、接口联调）

承担角色：前端工程师，负责状态管理（React Query）、接口联调、错误处理；维护 Developer + vercel-react-best-practices

主要挑战：

待补充

解决方法：

待补充

收获与反思：

待补充

5.4 项方灿 — 前端工程师（样式、响应式、可用性）

承担角色：前端工程师，负责样式系统、响应式适配、可用性优化；维护 Developer + vercel-react-best-practices

主要挑战：

待补充

解决方法：

待补充

收获与反思：

待补充

5.5 王建皓 — 后端工程师（鉴权、会话）

承担角色： 后端工程师，负责登录、注册、鉴权、会话管理（Spring Security + JWT）；维护 Developer + springboot-patterns

主要挑战：

待补充

解决方法：

待补充

收获与反思：

待补充

5.6 李明睿 — 后端工程师（帖子、评论、治理）

承担角色： 后端工程师，负责帖子、评论、内容治理接口；维护 Developer + springboot-patterns

主要挑战：

待补充

解决方法：

待补充

收获与反思：

待补充

5.7 罗新鹏 — 后端工程师（数据建模、Flyway）

承担角色： 后端工程师，负责数据模型（JPA Entity + Flyway 迁移）、校验与异常处理；维护 Developer + java-springboot

主要挑战：

待补充

解决方法:

待补充

收获与反思:

待补充

5.8 朱城弘 — 测试工程师 (后端 JUnit 5 + MockMvc)

承担角色: 测试工程师, 负责测试用例设计、后端接口测试 (JUnit 5 + MockMvc) ; 维护 QA 智能体 + qa.md 规则

主要挑战:

待补充

解决方法:

待补充

收获与反思:

待补充

5.9 陈瑜凡 — 测试工程师 (前端 Vitest + RTL)

承担角色: 测试工程师, 负责前端组件测试 (Vitest + RTL) 、回归验证; 维护 QA + qa.md

主要挑战:

待补充

解决方法:

待补充

收获与反思：

待补充

5.10 陈元煦 — DevOps 工程师（容器化、构建发布）

承担角色：DevOps 工程师，负责环境配置、构建发布、数据库运维、Docker 容器化；维护自动化脚本

主要挑战：

待补充

解决方法：

待补充

收获与反思：

待补充

5.11 王帆 — 安全与审查工程师（OWASP、XSS 防护）

承担角色：安全与审查工程师，负责安全基线审查、代码审查、知识沉淀；维护 CLAUDE.md 审查清单 + 安全规则

主要挑战：

待补充

解决方法：

待补充

收获与反思：

待补充

六、技术维度深度分析

6.1 压力测试

6.1.1 测试方法

使用 Python 3.11 标准库 (`concurrent.futures` + `urllib`) 编写压测脚本，零第三方依赖。

6.1.2 测试配置

参数	值
并发线程	50
持续时间	15 秒 / 固定请求数 (500-1000)
目标服务	Gateway (port 9000)
数据规模	10,005 帖 + 90,014 评论 + 16 用户
测试脚本	<code>docs/stress-test/stress_test.py</code>
测试日期	2026-06-14

6.1.3 测试结果

场景	总请求	成功率	RPS	平均延迟 (ms)	P95(ms)	P99(ms)
只读基准 (categories + posts + announcements)	545	100%	34.86	1394.18	3175.86	4234.13
单接口 GET /api/categories	1009	100%	64.83	754.96	1253.21	1809.74
单接口 GET /api/posts	858	99.65%	28.56	935.90	1326.45	1836.19
单接口 GET /api/announcements	2175	100%	143.32	343.79	669.83	955.04
认证登录 POST /api/auth/login	623	100%	39.19	1236.72	2108.13	2386.25
读写混合 80/20	1000	100%	92.64	527.14	976.86	1811.74
并发写入 POST /api/posts	500	100%	119.92	402.43	742.50	1019.20
帖子详情 + 评论	1982	100%	129.21	380.65	574.37	757.19

6.1.4 瓶颈与优化方向

- **P0 瓶颈**: BCrypt 密码验证 (平均 1237ms) , 可通过登录限流 + Redis Session 缓存优化
- **P1 瓶颈**: 帖子列表查询 (GET /api/posts) , 万级数据下出现 3 次 30s 超时, 建议为 `posts(status, created_at)` 建立复合索引或引入分页缓存
- **优化方向**: 引入 Redis 缓存 (categories、announcements 等字典数据) 、异步化浏览量更新、数据库连接池调优

详见: [docs/stress-test/stress-test-report.md](#)

6.2 微服务架构

6.2.1 拆分策略

采用 DDD (领域驱动设计) 的限界上下文方法, 按业务域拆分:

服务	端口	职责	数据表
Eureka Server	8761	服务注册与发现	—
API Gateway	9000	路由转发、负载均衡、重试	—
User Service	8081	认证、用户管理、密码重置	users, user_profiles, refresh_tokens, verification_codes
Forum Service	8082	帖子、评论、分类、点赞、收藏	posts, comments, categories, post_likes, bookmarks
Notification Service	8083	通知管理与推送	notifications
Content Service	8085	公告、举报、内容审核	announcements, reports
File Service	8086	文件上传与存储	文件系统

6.2.2 服务间通信

调用方	被调方	方式	用途
Forum	User	OpenFeign	获取帖子/评论作者信息
Forum	Notification	OpenFeign	评论时创建通知
Content	User	OpenFeign	公告作者信息
Content	Notification	OpenFeign	公告发布通知

6.2.3 收益分析

维度	单体	微服务
代码定位	98 个文件中搜索	10-20 个文件中定位
修改影响	全局回归	单服务测试
独立扩容	不可	按服务扩缩
故障隔离	单点全局不可用	降级后核心功能正常

6.2.4 过渡方案

当前采用 **共享数据库模式**（所有服务共用 `cc91_db`，通过表名隔离），作为 DB-per-Service 的过渡形态。未来可逐步拆分为独立数据库，引入 Saga 模式处理分布式事务。

详见：[docs/microservices-design.md](#)

6.3 系统监控

6.3.1 监控架构



6.3.2 工具选型

组件	选型	用途
指标暴露	Spring Boot Actuator + Micrometer	每个服务暴露 <code>/actuator/prometheus</code>
指标采集	Prometheus v2.48.0	每 15 秒拉取指标
可视化	Grafana 10.2.0	仪表盘展示
告警	Prometheus AlertManager	5 条告警规则

6.3.3 监控指标

类别	指标	用途
HTTP	<code>http_server_requests_seconds_*</code>	响应时间、吞吐、错误率
JVM	<code>jvm_memory_used_bytes</code>	堆内存使用
JVM	<code>jvm_threads_live_threads</code>	线程数
连接池	<code>hikaricp_connections_active</code>	数据库连接池
断路器	<code>resilience4j_circuitbreaker_*</code>	熔断状态

6.3.4 告警规则

告警	条件	级别
ServiceDown	<code>up == 0</code> 持续 1 分钟	Critical
HighErrorRate	5xx 错误率 > 10%	Warning
HighLatency	P95 > 2s	Warning
CircuitBreakerOpen	断路器打开	Warning
HighMemoryUsage	堆内存 > 85%	Warning

详见: `docs/monitoring-design.md`

6.4 弹性设计

6.4.1 已落地的弹性措施

措施	实现方式	效果
断路器	Resilience4j CircuitBreaker	服务不可用时熔断，10s 后半开探测
重试机制	Gateway spring-retry + LoadBalancer retry	服务启动期间自动重试，避免 503
降级处理	Feign Fallback	通知发送失败时记录日志，不影响主流程
负载均衡	Spring Cloud LoadBalancer + Eureka	多实例时自动 Round Robin
连接池隔离	HikariCP 每服务独立连接池	单服务数据库压力不传染其他服务
容器重启	Docker <code>restart: unless-stopped</code>	服务异常退出自动恢复
健康检查	Spring Boot Actuator + Docker healthcheck	MySQL、Eureka 就绪后才启动下游

6.4.2 可引入的进阶弹性策略

策略	适用场景	推荐方案
舱壁隔离	防止某接口耗尽线程池	Resilience4j Bulkhead，为不同 API 分配独立线程池
限流	防止暴力破解登录	Bucket4j 令牌桶，登录接口 10 次/分钟
服务降级	论坛浏览不受通知服务影响	通知不可用时返回空列表，不阻塞帖子加载
异步解耦	通知、浏览量更新	RabbitMQ 消息队列，削峰填谷
分布式追踪	排查跨服务调用延迟	Micrometer Tracing + Zipkin

6.5 安全性保障

6.5.1 安全措施总览（按 OWASP Top 10）

OWASP	类别	措施	状态
A01	权限控制失效	RBAC 角色控制 + @PreAuthorize + InternalApiAuthFilter	✓
A02	加密机制失败	BCrypt 密码哈希, JWT HMAC-SHA 签名	✓
A03	注入攻击	JPA 参数化查询, DOMPurify + HtmlSanitizer 双重 XSS 防护	✓
A04	不安全设计	无状态 JWT, DTO 与 Entity 分离	✓
A05	安全配置错误	环境变量管理密钥, 无硬编码, CORS 白名单	✓
A06	过时组件	mvn dependency-check:check + npm audit	✓
A07	认证失败	双 Token 机制, 账户锁定 (多次失败触发), Refresh Token 撤销	✓
A08	数据完整性	Flyway 版本化迁移, 数据库 schema 可追溯	✓
A09	日志失败	异常全量记录, 登录失败 WARN 日志	✓
A10	SSRF	服务间调用使用内部 API + INTERNAL_TOKEN 认证	✓

6.5.2 XSS 防护实战

前后端双重净化： - 前端：DOMPurify 对 dangerouslySetInnerHTML 内容白名单过滤 - 后端：HtmlSanitizer.java 移除 <script>、onXXX 事件、javascript: 协议

6.5.3 文件上传安全

防护层	措施
Content-Type	仅允许 image/jpeg、image/png、image/webp
扩展名	白名单 .jpg/.jpeg/.png/.webp
大小限制	Spring 2MB + Service 层二次校验
路径遍历	过滤 ..、/、\ 字符
重命名	{userId}_{timestamp}.{ext} 格式

6.5.4 已知风险与缓解

风险	等级	缓解
CORS 开发配置	中	生产部署前更新为实际域名
文件本地存储	低	单实例 OK，生产建议迁移 OSS/S3
邮件服务依赖	低	不影响核心功能，控制台可输出验证码

详见：`docs/SECURITY_AUDIT.md`

七、配套文档索引

7.1 核心设计文档

文档	路径	说明
微服务设计	<code>docs/microservices-design.md</code>	拆分策略、通信机制、收益分析
监控设计	<code>docs/monitoring-design.md</code>	Prometheus + Grafana 方案
安全审查	<code>docs/SECURITY_AUDIT.md</code>	OWASP Top 10 全覆盖审查
压力测试报告	<code>docs/stress-test/stress-test-report.md</code>	8 场景性能数据
API 参考	<code>docs/api-reference.md</code>	接口文档
环境搭建	<code>docs/ENV_SETUP.md</code>	本地开发环境指南
验收指南	<code>docs/acceptance-guide.md</code>	答辩/验收操作指南
Docker 部署	<code>docs/docker-deployment-guide.md</code>	容器化一键部署

7.2 智能体配置

文件	说明
CLAUDE.md	Lead 工作协议，完整 5 阶段工作流
.claude/agents/developer.md	Developer Agent 配置（技能、职责、自查清单）
.claude/agents/qa.md	QA Agent 配置（测试红线、质量标准）
.claude/settings.json	项目级 Claude Code 配置（MCP 接入）
.claude/settings.local.json	本地覆盖配置

7.3 技能定义

Skill	路径	说明
java-springboot	.claude/skills/java-springboot/SKILL.md	Spring Boot 最佳实践
springboot-patterns	.claude/skills/springboot-patterns/SKILL.md	分层架构模式
typescript-react-reviewer	.claude/skills/typescript-react-reviewer/SKILL.md	React 代码审查（含反模式、React 19 模式、检查清单）
vercel-react-best-practices	.claude/skills/vercel-react-best-practices/	60+ 条性能优化规则
web-design-guidelines	.claude/skills/web-design-guidelines/SKILL.md	Web 界面设计规范

7.4 测试与数据

文件	说明
docs/stress-test/stress_test.py	Python 压力测试脚本（零依赖）
docs/stress-test/stress_test_result.json	压力测试原始数据
docs/stress-test/seed_data.sql	万级种子数据脚本
docs/unit-test-report.md	后端单元测试报告
docs/integration-test-report.md	集成测试报告
frontend/src/__tests__/	前端 46 个测试文件（Vitest + RTL）
各服务 src/test/	后端单元测试（JUnit 5 + MockMvc）

7.5 项目配置

文件	说明
docker-compose.yml	9 个容器编排（MySQL + 7 服务 + Prometheus + Grafana）
.env / .env.example	环境变量（密码、密钥）
scripts/build.ps1	一键编译脚本
nginx/nginx.conf	前端 Nginx 反向代理配置
monitoring/	Prometheus + Grafana 配置

八、项目亮点总结

- 1. **微服务架构落地**：7 个微服务 + Gateway + Eureka，按业务域拆分，服务间通过 OpenFeign + Resilience4j 通信
- 2. **智能开发工具链**：Claude Code + CodeGraph MCP Server + Playwright MCP + 5 个专业 Skills 形成完整 AI 辅助开发体系
- 3. **全链路安全防护**：OWASP Top 10 全覆盖，XSS 前后端双重净化，文件上传 5 层校验
- 4. **可观测性**：Prometheus + Grafana 全链路监控，5 条告警规则，JVM/HTTP/DB 全维度指标
- 5. **容器化部署**：Docker Compose 一键启动 9 个容器，healthcheck 保证启动顺序
- 6. **分层测试**：前端 46 个测试文件 + 后端分层测试 + 压力测试 8 场景，关键业务路径全覆盖
- 7. **文档体系完整**：架构、安全、性能、验收、API 五大类文档，可交付评审